

Dokumen Serah Terima Infrastruktur (Handover Document) - FP TKA

Kelompok A3

Dokumen ini merangkum status terkini infrastruktur cloud dan memberikan panduan teknis yang spesifik untuk fase selanjutnya: Optimasi Database dan Deployment Frontend. Dokumen ini ditujukan untuk anggota tim yang bertanggung jawab pada peran tersebut.

1. Status Infrastruktur Saat Ini (Checklist DevOps)

Infrastruktur dasar telah dikonfigurasi dan diamankan. Berikut adalah rekapitulasi statusnya:

Komponen	Alamat IP (Publik)	Alamat IP (Privat/VPC)	Status	Keterangan
Load Balancer	129.212.209.53	-	Selesai	Menerima traffic di port 80 dan mendistribusikannya ke worker.
Worker-1	168.144.45.189	10.104.0.3	Selesai	Backend (Flask+Gunicorn) aktif. Terkoneksi ke VPC. Firewall diatur hanya menerima port 80 dari Load Balancer dan

Komponen	Alamat IP (Publik)	Alamat IP (Privat/VPC)	Status	Keterangan
				port 22 dari IP pengelola.
Worker-2	168.144.136.155	10.104.0.4	Selesai	Backend aktif. Terkoneksi ke VPC. Firewall terkonfigurasi.
Worker-3	152.42.191.158	10.104.0.5	Selesai	Backend aktif. Terkoneksi ke VPC. Firewall terkonfigurasi.
Database (MongoDB)	68.183.185.9	10.104.0.2	Selesai (Infra)	MongoDB 7.0 terinstal dan berjalan. `bindIp` diset ke `0.0.0.0`. Firewall diatur hanya menerima koneksi port 27017 dari IP Privat Worker 1, 2, dan 3.

2. Panduan Tugas Lanjutan: Backend & DB Optimization

Infrastruktur database sudah siap menerima koneksi, namun data belum dimasukkan dan performa kueri belum dioptimalkan. Anggota tim dengan peran ini harus mengeksekusi langkah-langkah berikut secara berurutan pada server database.

Langkah 2.1: Akses Server Database

Semua eksekusi dilakukan di dalam server db-mongo. Gunakan SSH untuk mengaksesnya melalui terminal lokal Anda.

```
ssh root@68.183.185.9
```

Langkah 2.2: Restore Data Dump

Data awal harus dimasukkan ke dalam MongoDB. Pastikan file dump (biasanya berekstensi .bson atau .json, atau dalam bentuk folder hasil ekspor) sudah tersedia di server db-mongo. Gunakan perintah mongorestore untuk mengimpor data.

```
mongorestore --drop --db <nama_database_kalian>  
<path_ke_folder_dump>
```

Penjelasan: Perintah ini mengembalikan (restore) data dari backup. Flag --drop berguna untuk menghapus koleksi yang sudah ada sebelumnya (jika ada) sebelum mengimpor data baru, memastikan data bersih. Ganti <nama_database_kalian> dengan nama spesifik database project, dan <path_ke_folder_dump> dengan lokasi direktori data dump.

Langkah 2.3: Konfigurasi Indexing (Krusial untuk Performa)

Tanpa indexing, MongoDB akan memindai setiap dokumen dalam koleksi saat menerima kueri (Collection Scan), yang akan menyebabkan server kelebihan beban saat load testing. Indexing membuat pencarian jauh lebih efisien.

1. Masuk ke dalam shell interaktif MongoDB:

```
mongosh
```

2. Pilih database yang baru saja di-restore:

```
use <nama_database_kalian>
```

3. Buat index berdasarkan pola kueri yang akan digunakan oleh aplikasi. Berikut adalah contoh pembuatan index berdasarkan field yang sering diakses:

```
// Mengasumsikan koleksi bernama 'users' dan pencarian sering  
menggunakan 'email'
```

```
db.users.createIndex({ "email": 1 })
```

```
// Mengasumsikan koleksi bernama 'orders' dan sering diurutkan  
berdasarkan waktu pembuatan
```

```
db.orders.createIndex({ "created_at": -1 })
```

```
// Mengasumsikan pencarian spesifik menggunakan ID order
```

```
db.orders.createIndex({ "order_id": 1 })
```

Penjelasan: 1 berarti index diurutkan secara ascending (A-Z/kecil ke besar), dan -1 berarti descending. Sesuaikan field (email, created_at, order_id) dengan struktur data sebenarnya.

4. Verifikasi bahwa index telah berhasil dibuat:

```
db.<nama_koleksi>.getIndexes()
```

5. Keluar dari shell MongoDB:

```
exit
```

3. Panduan Tugas Lanjutan: Deployment Frontend

Setelah backend dan database stabil, antarmuka pengguna (Frontend) harus disiapkan agar dapat berkomunikasi dengan sistem melalui Load Balancer.

Langkah 3.1: Konfigurasi API Endpoint di Kode Frontend

Sebelum memindahkan file frontend ke server, kode sumber (seperti file JavaScript utama atau konfigurasi environment) harus diubah agar merujuk ke Load Balancer, bukan ke localhost atau IP spesifik worker.

Cari variabel konfigurasi (biasanya bernama API_BASE_URL, REACT_APP_API_URL, atau serupa) di proyek frontend lokal Anda, dan ubah nilainya menjadi IP publik Load Balancer.

```
// Contoh perubahan di file config.js atau environment variable  
const API_BASE = "http://129.212.209.53";
```

Penjelasan: Ini memastikan semua permintaan data dari peramban pengguna akan dikirim ke Load Balancer, yang kemudian akan mendistribusikannya ke worker yang paling sedikit beban

kerjanya.

Langkah 3.2: Build Frontend

Jika menggunakan framework seperti React, Vue, atau Angular, Anda harus mengompilasi kode menjadi file statis (HTML, CSS, JS) yang siap untuk di-deploy.

```
npm run build
```

Penjelasan: Perintah ini akan menghasilkan sebuah folder (biasanya bernama build atau dist) yang berisi file yang telah dioptimasi.

Langkah 3.3: Deployment ke DigitalOcean Spaces (Sangat Direkomendasikan)

Metode terbaik untuk aplikasi SPA (Single Page Application) adalah menggunakan layanan Object Storage (seperti DO Spaces) karena lebih murah, sangat cepat, dan mengurangi beban server backend.

1. Buka DigitalOcean Dashboard.
2. Buka menu **Spaces**.
3. Pilih Space yang telah dibuat (atau buat baru jika belum ada).
4. Buka tab **Settings** pada Space tersebut.
5. Cari opsi **File Sharing / CDN Endpoint** dan pastikan CDN telah diaktifkan untuk mempercepat pengiriman file.
6. Cari pengaturan **Static Site Hosting** dan aktifkan. Tetapkan index.html sebagai *Index Document* dan juga sebagai *Error Document* (sangat penting untuk routing SPA).
7. Unggah semua file dan folder yang berada **di dalam** folder build/dist (bukan foldernya itu sendiri) ke root directory Space Anda. Pastikan semua file diatur permissions-nya menjadi *Public*.
8. Akses aplikasi melalui URL endpoint Static Site yang disediakan oleh Spaces.

Alternatif Deployment (Nginx di Worker)

Jika tidak menggunakan Spaces, frontend statis dapat ditempatkan di server Nginx yang ada di salah satu worker (meskipun tidak disarankan untuk arsitektur terdistribusi seperti ini). Anda perlu mentransfer file build ke server via SCP/SFTP dan mengubah konfigurasi Nginx (/etc/nginx/sites-available/default) untuk melayani folder statis tersebut, memastikan trafik API diteruskan (proxy_pass) ke backend Flask.